

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина: Архитектура вычислительных систем

ОТЧЕТ
к лабораторной работе №3
на тему

**ПРОГРАММИРОВАНИЕ
МНОГОЯДЕРНЫХ
АРХИТЕКТУР**

Выполнил:
студент гр. 153503
Кахновский Е.С.

Проверил:
Калиновская А.А.

Минск 2023

СОДЕРЖАНИЕ

1 Цель работы	3
2 Теоретические сведения	4
3 Результат выполнения	5
Заключение	6
Приложение А (обязательное) Листинг кода.....	7

1 ЦЕЛЬ РАБОТЫ

Использование интерфейса *OpenMP* для программирования простых многопоточных приложений.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

OpenMP – интерфейс прикладного программирования (API) для масштабируемых *SMP*-систем в модели общей памяти. Исполняемый процесс в памяти может состоять из множественных нитей, которые имеют общее адресное пространство, но разные потоки команд и отдельные стеки. В простейшем случае, процесс состоит из одной нити, выполняющую функцию *main*. Нити иногда называют также потоками, легковесными процессами, *LWP*. *OpenMP* основан на существовании множественных потоков в общедоступной памяти. Схема процесса представлена на рисунке 1.

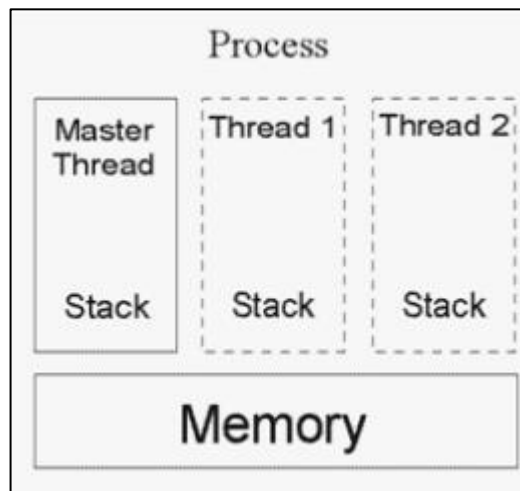


Рисунок 1

Все программы *OpenMP* начинаются как единственный процесс с главным потоком, который выполняется последовательно, пока не сталкиваются с первой областью параллельной конструкции. Создание нескольких потоков (*FORK*) и объединение (*JOIN*) находится на рисунке 2.

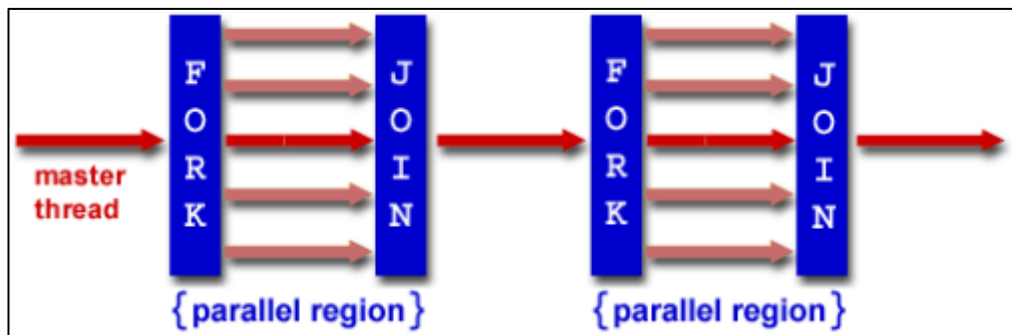
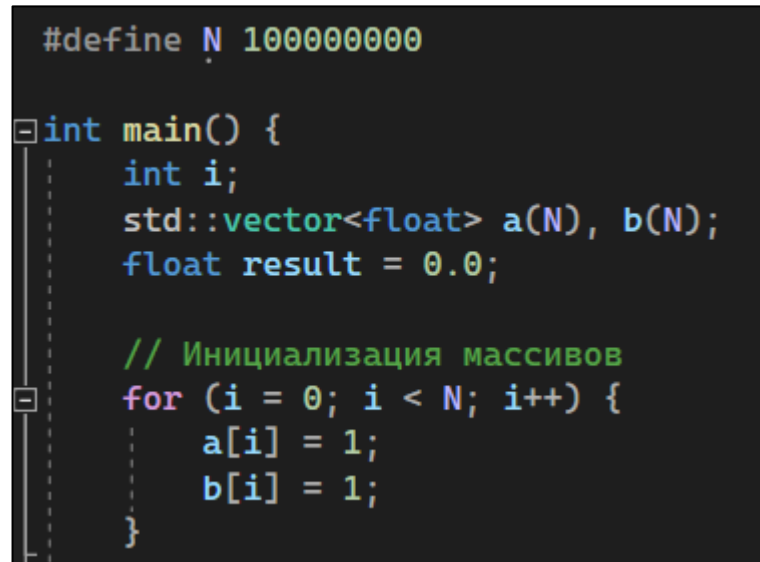


Рисунок 2

3 РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

В соответствии с вариантом №1 была реализована программа для вычисления скалярного произведения двух векторов.

Инициализация данных задается в коде программы (рисунок 1).

A screenshot of a code editor showing C++ code. The code defines a constant N as 100000000. In the main function, it declares an integer i, two std::vector<float> objects a and b of size N, and a float result initialized to 0.0. A comment in Russian indicates the initialization of arrays. A for loop iterates from i = 0 to i < N, setting a[i] = 1 and b[i] = 1 for each iteration. The code is color-coded: keywords in blue, comments in green, and literals in yellow.

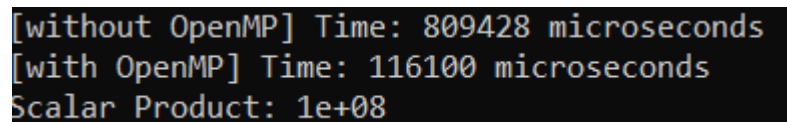
```
#define N 100000000

int main() {
    int i;
    std::vector<float> a(N), b(N);
    float result = 0.0;

    // Инициализация массивов
    for (i = 0; i < N; i++) {
        a[i] = 1;
        b[i] = 1;
    }
}
```

Рисунок 1 – Инициализация данных

После выполнения, программа выводит результат скалярного произведения и время, затраченное на подсчет без использования OpenMP и с использованием. (рисунок 2).

A screenshot of a terminal window showing the output of the program. It displays the execution time in microseconds for two cases: without OpenMP (809428) and with OpenMP (116100). It also shows the scalar product result as 1e+08.

```
[without OpenMP] Time: 809428 microseconds
[with OpenMP] Time: 116100 microseconds
Scalar Product: 1e+08
```

Рисунок 2 – Результат выполнения программы

Можем заметить, что на достаточно большом количестве элементов наблюдается значительный прирост скорости вычисления с использованием OpenMP.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы было изучено использование интерфейса OpenMP для программирования простых многопоточных приложений. Наглядно было замечено, что использование OpenMP позволяет достаточно сильно сократить время работы приложения при достаточно больших объемах данных.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг кода

```
#include <iostream>
#include <omp.h>
#include <chrono>
#include <vector>

#define N 1000000000

int main() {
    int i;
    std::vector<float> a(N), b(N);
    float result = 0.0;

    // Инициализация массивов
    for (i = 0; i < N; i++) {
        a[i] = 1;
        b[i] = 1;
    }

    // Измерение времени выполнения с использованием chrono
    auto start_time = std::chrono::high_resolution_clock::now();
    // Вычисление скалярного произведения без использования OpenMP
    for (i = 0; i < N; i++) {
        result += a[i] * b[i];
    }
    auto end_time = std::chrono::high_resolution_clock::now();
    auto duration =
std::chrono::duration_cast<std::chrono::microseconds>(end_time -
start_time).count();

    // Вывод результата и времени выполнения без OpenMP
    std::cout << "[without OpenMP] Time: " << duration << " microseconds" <<
std::endl;
    // Измерение времени выполнения с использованием chrono
    auto start_time_omp = std::chrono::high_resolution_clock::now();

    result = 0;
    // Вычисление скалярного произведения с использованием OpenMP
#pragma omp parallel for reduction(+:result)
    for (i = 0; i < N; i++) {
        result += a[i] * b[i];
    }
    auto end_time_omp = std::chrono::high_resolution_clock::now();
    auto duration_omp =
std::chrono::duration_cast<std::chrono::microseconds>(end_time_omp -
start_time_omp).count();

    // Вывод результата и времени выполнения
    std::cout << "[with OpenMP] Time: " << duration_omp << " microseconds" <<
std::endl;
    std::cout << "Scalar Product: " << result << std::endl;

    return 0;
}
```